UNIVERSITI TEKNOLOGI MARA
MARKING SCHEME

COURSE	:	COMPUTER ORGANIZATION AND ARCHITECTURE
COURSE CODE	:	CSC429
EXAMINATION	:	NOVEMBER 2026 (MOCK)
TIME	:	3 HOURS

MARKING SCHEME NOTES

1. This marking scheme accompanies the CSC429 mock paper (November 2026).
   Answers are unofficial reference answers, not official UTM marking schemes.

2. Marking points show the expected answer and mark allocation; check them against the source paper.

3. Part A = 4 x 5 + 6 x 10; Part B = 1 x 20; total = 100 marks.

4. Approved real questions only; no generators, placeholders or parametric variants.
   Questions and answers are for local study use only.

5. Structure derived from the supplied CSC429 papers; every question remains traceable to its source page.

FOR MARKERS' USE ONLY

This marking scheme consists of 9 printed pages

© Hak Cipta Universiti Teknologi MARA

PART A — MARKING GUIDE

QUESTION 1 (5 marks)

Source: CSC429-2018-12, page [5]

Unofficial model answer:

The question asks for a suitable example for each processor organization. In CSC429 the standard taxonomy of processor organizations is based on the number of instruction streams and data streams (Flynn's classification), so a suitable example is given for each of the four classes. SISD (Single Instruction stream, Single Data stream): a conventional single-core scalar processor, e.g. a basic single-core x86/Pentium or an ARM Cortex-M microcontroller executing one instruction on one data item per cycle. SIMD (Single Instruction stream, Multiple Data streams): a vector processor or GPU/array processor, e.g. a GPU (CUDA cores) or Intel SSE/AVX vector instructions applying the same add to several packed data elements, or an array processor such as the ILLIAC IV. MISD (Multiple Instruction streams, Single Data stream): no widely built commercial machine; the usual textbook example is a systolic array or a fault-tolerant system where several processors apply different operations to the same data stream (e.g. a pipeline of processors on one data stream). MIMD (Multiple Instruction streams, Multiple Data streams): a multicore/multiprocessor system, e.g. a multi-core CPU (Intel Core i7) or a cluster of workstations, where each core executes its own instruction stream on its own data. If the lecturer's 'processor organization' means the internal CPU organization instead, the examples are: single-cycle processor (e.g. a simple MIPS single-cycle datapath), multi-cycle processor (e.g. a MIPS multi-cycle datapath using one ALU and registers), and pipelined processor (e.g. a 5-stage MIPS pipeline).

Marking points:

- *1 mark: SISD correctly defined and given a valid example (single-core scalar CPU/microcontroller)*
- *1 mark: SIMD correctly defined and given a valid example (vector processor, GPU, SSE/AVX, array processor)*
- *1 mark: MISD correctly defined and given a valid example (systolic array / pipeline of processors on one data stream), noting it is rare*
- *1 mark: MIMD correctly defined and given a valid example (multicore CPU or multiprocessor/cluster)*
- *1 mark: clear, consistent classification scheme (Flynn) and one example per organization, with correct terminology*

Model confidence: medium.

Model note: The question text does not specify which taxonomy of 'processor organization' is intended; the answer assumes Flynn's SISD/SIMD/MISD/MIMD classification, which is the standard CSC429 treatment, and also notes the single-cycle/multi-cycle/pipelined alternative in case the lecturer meant internal CPU organization. No figure or table was supplied, so examples are generic textbook examples rather than ones tied to a specific diagram.

QUESTION 2 (5 marks)

Source: CSC429-2017-07, page [5]

Unofficial model answer:

a) Two advantages of SMP organization: (1) Scalability and cost-effectiveness — additional processors can be added to the shared bus/memory to increase throughput, and commodity processors are used, giving good price/performance. (2) Simplified programming and load balancing — all processors share a single physical address space and see the same memory, so a single OS image can schedule threads across processors without explicit message passing, and workload can be balanced dynamically among processors.

b) Cluster organization: A cluster is a set of independent, complete computer systems (nodes, each with its own processor, memory, and OS) interconnected by a high-speed network (e.g., LAN or dedicated interconnect) and cooperating as a single system under cluster middleware. Each node retains its own private memory; communication is by message passing, and the cluster may be configured for high availability (failover) or high-performance parallel computing.

© Hak Cipta Universiti Teknologi MARA

c) One benefit of cluster organization: High availability/fault tolerance — if a node fails, another node can take over its work (failover), so the system continues to provide service; alternatively, excellent scalability/price-performance by adding commodity nodes. (Any one benefit is acceptable.)

Marking points:

• 1 mark: first valid advantage of SMP (e.g., scalability/cost-effectiveness with commodity processors)

• 1 mark: second valid advantage of SMP (e.g., single shared memory/single OS image simplifies programming and load balancing)

• 2 marks: definition of cluster organization — independent computers/nodes interconnected by a high-speed network, each with own memory/OS, cooperating as a single system (award 1 mark for partial: e.g., only 'group of computers connected by network' without independence/cooperation)

• 1 mark: one valid benefit of cluster organization (e.g., high availability/fault tolerance, scalability, price-performance)

Model confidence: high.

Model note: Unofficial model answer; not an official UTM marking scheme. No figure/table was supplied. Part (a) asks for two advantages; part (b) definition; part (c) one benefit. Marks allocated as 2+2+1=5.

QUESTION 3 (5 marks)

Source: CSC429-2026-02, page [2]

Unofficial model answer:

a) The integrated circuit (IC) is credited to Jack Kilby (Texas Instruments, 1958), who demonstrated the first working IC, with Robert Noyce (Fairchild, 1959) independently developing a practical silicon version; the usual exam answer is Jack Kilby.

b) The integrated circuit was first introduced in the third generation of computers (1964–1971), replacing discrete transistors and ushering in machines such as the IBM System/360.

c) Three main components of an integrated circuit are: (1) the active semiconductor devices — transistors (and diodes) formed on the silicon die; (2) passive components — resistors and capacitors (and their interconnections); and (3) the interconnection wiring/metallization and the semiconductor substrate (chip) on which they are fabricated, together with the package and bonding pads that connect the die to external pins.

Marking points:

• 1 mark (a): Jack Kilby (accept Robert Noyce as co-inventor; do not accept unrelated names).

• 1 mark (b): Third generation.

• 3 marks (c): 1 mark each for any three valid IC components, e.g. transistors, diodes, resistors, capacitors, interconnection wiring/metallization, semiconductor substrate/die, package/bonding pads.

Model confidence: high.

Model note: No figure or table was supplied; the answer is based on standard CSC429 computer-generation material. Some textbooks credit Noyce alongside Kilby, so either name should be accepted for (a).

QUESTION 4 (5 marks)

Source: CSC429-2014-12, page [3]

Unofficial model answer:

a) Speed: SRAM is faster than DRAM. SRAM access times are typically on the order of a few nanoseconds (e.g., 1–10 ns), while DRAM access times are longer (e.g., 10–100 ns) due to the need to sense and restore charge. SRAM is used for cache memory; DRAM for main memory.

b) Size per bit: SRAM uses more transistors per bit (typically 6 transistors per bit, forming a flip-flop), so it occupies a larger chip area per bit. DRAM uses one transistor plus one capacitor per bit, giving a much smaller size per bit and higher density.

c) Cost: SRAM is more expensive per bit than DRAM because of its larger cell size and lower density. DRAM

© Hak Cipta Universiti Teknologi MARA

is cheaper per bit and is therefore used for large-capacity main memory.

d) Type of bits stored: SRAM stores bits as the state of a bistable flip-flop (voltage levels held by cross-coupled inverters), so the stored value is retained as long as power is applied. DRAM stores bits as electric charge on a capacitor; the presence or absence of charge represents the bit value.

e) Refreshing need when powered: SRAM does not require refreshing while powered; the flip-flop holds its state indefinitely as long as power is supplied. DRAM requires periodic refreshing (typically every few milliseconds) because the capacitor charge leaks away, even when powered.

Marking points:

• 1 mark: Speed – SRAM faster than DRAM (with brief reason such as no refresh/sense-restore overhead).

• 1 mark: Size per bit – SRAM larger cell (e.g., 6 transistors) vs DRAM smaller cell (1 transistor + 1 capacitor).

• 1 mark: Cost – SRAM more expensive per bit than DRAM.

• 1 mark: Type of bits stored – SRAM uses flip-flop/latch state; DRAM uses capacitor charge.

• 1 mark: Refreshing need when powered – SRAM no refresh needed; DRAM needs periodic refresh.

Model confidence: high.

Model note: Unofficial model answer; not an official UTM marking scheme. No figure or table was supplied. Typical transistor counts and refresh intervals are illustrative and not required for full marks.

QUESTION 5 (10 marks)

Source: CSC429-2025-02, page [4]

Unofficial model answer:

a) System bus timing approaches: Synchronous timing uses a fixed clock and predetermined time slots for bus operations; all devices must operate at the same clock rate, and the bus cycle length is fixed. Example: a synchronous bus where the processor places an address on the bus at the rising edge of the clock and data is transferred at a fixed later clock edge (e.g., a simple memory read cycle with a fixed number of wait states). Asynchronous timing uses handshaking signals (e.g., request/acknowledge) rather than a clock; each bus operation begins when the master asserts a request and completes when the slave returns an acknowledge, so devices of different speeds can interoperate. Example: an asynchronous bus using a 'ready' or 'acknowledge' line where a slow memory device delays the completion of the cycle until it is ready. In summary, synchronous timing is simple and fast for matched-speed devices but inflexible, while asynchronous timing is flexible and accommodates heterogeneous device speeds at the cost of more complex control logic.

b) One reason an I/O module is needed: to provide a standardized interface between the system bus and a wide variety of external devices that differ in data format, speed, and control methods. The I/O module performs functions such as buffering, format conversion, error detection, and control/timing, so the processor does not need to manage device-specific details directly. This decouples the CPU from peripheral characteristics and allows many devices to be connected through a common bus interface.

c) Two interfaces to external devices: (1) parallel interface (e.g., a parallel port or parallel bus such as ATA/IDE) where multiple bits are transferred simultaneously over separate lines; (2) serial interface (e.g., RS-232, USB, or SATA) where data is transferred one bit at a time over a single line (or differential pair). Other acceptable examples include SCSI, FireWire, or a network interface, but the two listed are standard.

Marking points:

• 1 mark: Correctly describes synchronous timing (fixed clock, predetermined slots, all devices same speed).

• 1 mark: Gives an appropriate example of synchronous timing (e.g., fixed clock bus cycle with wait states).

• 1 mark: Correctly describes asynchronous timing (handshaking, request/acknowledge, no common clock).

• 1 mark: Gives an appropriate example of asynchronous timing (e.g., ready/acknowledge handshake with slow device).

• 1 mark: Compares the two approaches (e.g., simplicity/speed vs flexibility/complexity).

• 1 mark: States a valid reason for needing an I/O module (e.g., device heterogeneity, buffering, format conversion,

© Hak Cipta Universiti Teknologi MARA

decoupling CPU from device specifics).

• 1 mark: Explains the reason adequately (links to standardized interface or processor offloading).

• 1 mark: Lists one correct interface to external devices (e.g., parallel interface).

• 1 mark: Lists a second correct interface to external devices (e.g., serial interface).

• 1 mark: Provides a brief description or example for each interface to show understanding.

Model confidence: high.

Model note: This is an unofficial model answer, not an official UTM marking scheme. The question text does not include any figure or table; the answer is based solely on the provided text. Part (a) asks for comparison of two timing approaches; the answer assumes synchronous and asynchronous bus timing as the intended pair. Part (c) accepts any two valid external device interfaces; examples given are illustrative.

QUESTION 6 (10 marks)

Source: CSC429-2025-07, page [4]

Unofficial model answer:

a) SRAM stores each bit in a bistable latch (typically a 6-transistor cell: two cross-coupled inverters plus two access transistors). The cross-coupled inverters hold the bit as a stable state (Q and Q-bar); the access transistors connect the cell to the bitlines when the wordline is asserted, allowing read or write. DRAM stores each bit as charge on a capacitor (1T1C cell: one access transistor and one storage capacitor). A logic 1 is represented by charge stored on the capacitor and a logic 0 by little or no charge. Because the capacitor leaks, DRAM cells must be periodically refreshed (read and rewritten) to retain data. (Note: the question text says 'DFiAM'; this is taken to mean DRAM.)

b) SRAM is faster than DRAM because SRAM uses a bistable latch that drives the bitline actively during a read, so the bitline voltage swing develops quickly and no refresh or precharge-restore cycle is needed for retention. DRAM reads are destructive: the small charge on the capacitor must be sensed by a sensitive sense amplifier, then the cell must be rewritten (restored), and the bitline must be precharged before the next access. DRAM also has higher latency due to row activation, column access, and refresh overhead. Thus SRAM has lower access latency and higher speed.

c) Cost: SRAM is much more expensive per bit than DRAM because an SRAM cell uses about 6 transistors versus 1 transistor plus 1 capacitor for DRAM, so SRAM has lower density and larger area per bit. DRAM is therefore cheaper per bit and is used for main memory, while SRAM is used for caches. Power consumption: SRAM consumes relatively low static power (only leakage) when idle, but can consume dynamic power during active reads/writes. DRAM consumes static power due to refresh even when idle, and has higher active power from row activation and precharge; however, DRAM's per-bit power can be lower in some standby contexts. Overall, SRAM has lower idle/static power per cell but higher cost per bit; DRAM has refresh power overhead but lower cost per bit.

Marking points:

• 1 mark: SRAM cell description (bistable latch / cross-coupled inverters / 6T).

• 1 mark: DRAM cell description (capacitor storage / 1T1C).

• 1 mark: DRAM refresh requirement mentioned.

• 1 mark: SRAM read is non-destructive and actively driven.

• 1 mark: DRAM read is destructive and requires sense amplifier and restore.

• 1 mark: DRAM requires precharge/row activation overhead.

• 1 mark: SRAM cell size (≈6 transistors) vs DRAM (1T1C) leading to density difference.

• 1 mark: SRAM higher cost per bit, DRAM lower cost per bit.

• 1 mark: SRAM low static power (leakage only) when idle.

• 1 mark: DRAM refresh power overhead and/or active power comparison.

Model confidence: high.

Model note: The question text contains 'DFiAM', which is assumed to be a typo for DRAM. No figure or table was supplied. The

© Hak Cipta Universiti Teknologi MARA

answer is an unofficial model answer, not an official UTM marking scheme.

QUESTION 7 (10 marks)

Source: CSC429-2023-07, page [3]

Unofficial model answer:

a) To convert 273.78125_{10} to IEEE 754 single precision: sign = 0 (positive). Integer part 273 = 100010001_2. Fractional part 0.78125 = 0.11001_2 (0.78125×2=1.5625→1; 0.5625×2=1.125→1; 0.125×2=0.25→0; 0.25×2=0.5→0; 0.5×2=1.0→1). So $273.78125 = 100010001.11001_2$. Normalize: $1.0001000111001_2 \times 2^8$. Exponent field = 8 + 127 = 135 = 10000111_2. Mantissa (23 bits) = 00010001110010000000000. Significand bits: 0 10000111 00010001110010000000000. Group into hex: 0100 0011 1000 1000 1110 0100 0000 0000 = 0x4388E400. Thus IEEE single precision hexadecimal = 4388E400H.

b) C1588000H in IEEE single precision: binary = 1100 0001 0101 1000 1000 0000 0000 0000. Sign bit = 1 (negative). Exponent field = $10000010_2 = 130_{10}$; unbiased exponent = 130 − 127 = 3. Mantissa fraction = $10110001000000000000000_2$. Significand = 1.10110001_2 = 1 + 1/2 + 1/8 + 1/16 + 1/256 = 1.69140625. Value = −1.69140625 × 2^3 = −13.53125. Thus decimal value = −13.53125.

Marking points:

- *1 mark: correct sign bit 0 for positive number in (a)*
- *1 mark: correct binary integer part 100010001_2 for 273*
- *1 mark: correct binary fractional part 0.11001_2 for 0.78125*
- *1 mark: correct normalized form $1.0001000111001_2 \times 2^8$*
- *1 mark: correct biased exponent 135 = 10000111_2*
- *1 mark: correct 23-bit mantissa 00010001110010000000000*
- *1 mark: correct hexadecimal 4388E400H for (a)*
- *1 mark: correct sign bit 1 (negative) and exponent field 10000010_2 = 130 for (b)*
- *1 mark: correct significand 1.10110001_2 = 1.69140625*
- *1 mark: correct final decimal value −13.53125 for (b)*

Model confidence: high.

Model note: The question text uses '273.7812510' and 'C158 8000H'; these are interpreted as 273.78125_{10} and C1588000H respectively. No figure or table is required. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 8 (10 marks)

Source: CSC429-2018-12, page [5]

Unofficial model answer:

a) The keyboard belongs to the human-readable input/output module category. Human-readable devices are those that allow a user to communicate directly with the computer, typically through text or simple commands; the keyboard is an input-only human-readable device.

b) Two other devices in the same human-readable category are: (i) a computer monitor/display (human-readable output), and (ii) a printer (human-readable output). Other acceptable examples include a mouse or a touchscreen as human-readable input devices.

c) The basic unit of exchange for a keyboard is a character (typically an 8-bit byte representing an ASCII/Unicode character). Each key press is translated by the keyboard controller into a character code that is transferred to the I/O module.

d) Two types of characters are: (i) printable/displayable characters, such as letters, digits, punctuation marks and symbols (e.g., 'A', '7', '$'), which are intended to be shown on a display or printed; and (ii) control characters, such as carriage return (CR), line feed (LF), backspace (BS), escape (ESC) and tab (TAB), which do not produce a visible glyph but control the formatting or behaviour of the device or software.

© Hak Cipta Universiti Teknologi MARA

Marking points:

• 1 mark: (a) Correctly identifies keyboard as human-readable (input) category.

• 2 marks: (b) One mark for each correct device in the human-readable category (e.g., monitor, printer, mouse, touchscreen).

• 1 mark: (c) States character (or byte/ASCII character) as the basic unit of exchange.

• 6 marks: (d) Three marks per type of character: 1 mark for naming the type, 2 marks for a correct explanation/example. Accept printable vs control characters, or any other valid classification (e.g., alphanumeric vs special symbols) with justification.

Model confidence: medium.

Model note: The question does not define 'types of characters'; the answer assumes the standard printable vs control character distinction. Part (b) accepts any human-readable device; if the examiner intended only input devices, mouse/touchscreen are safer than monitor/printer. No official marking scheme is implied.

QUESTION 9 (10 marks)

Source: CSC429-2015-12, page [4]

Unofficial model answer:

The four access methods based on the hardware implementation of a storage device are: (1) Sequential access — data is read or written in a fixed linear order, one record after another; the access time depends on the position of the desired record relative to the current position. Example: magnetic tape drive. (2) Direct access — data is organised into blocks or records that can be accessed in any order, but the device still uses a shared read/write mechanism that must move to the target block; access time depends on the block location. Example: hard disk drive (HDD). (3) Random access — any storage location can be accessed directly in approximately the same amount of time, independent of the previous access; each location has a unique address and the hardware can select it immediately. Example: main memory (RAM) or solid-state drive (SSD). (4) Associative access — data is accessed by content rather than by address; the hardware compares a search key against stored tags in parallel and returns matching entries. Example: cache memory (associative cache) or content-addressable memory (CAM).

Marking points:

• 1 mark: Correctly naming sequential access and giving a valid example (e.g., magnetic tape).

• 1 mark: Correctly describing sequential access (linear order, access time depends on position).

• 1 mark: Correctly naming direct access and giving a valid example (e.g., hard disk drive).

• 1 mark: Correctly describing direct access (blocks/records accessed in any order but with a shared mechanism; access time depends on block location).

• 1 mark: Correctly naming random access and giving a valid example (e.g., main memory/RAM or SSD).

• 1 mark: Correctly describing random access (any location accessed in approximately equal time, independent of previous access).

• 1 mark: Correctly naming associative access and giving a valid example (e.g., cache memory or CAM).

• 1 mark: Correctly describing associative access (access by content, parallel comparison of tags).

• 1 mark: Clear organisation and use of correct terminology across all four methods.

• 1 mark: All four methods and examples are distinct and appropriate to hardware implementation.

Model confidence: high.

Model note: The question asks for four access methods based on hardware implementation; the standard set from Stallings is sequential, direct, random, and associative. Some textbooks may classify SSD as random access or as a separate category; the answer uses the standard classification. No figure or table was supplied, so no limitation from extracted text applies.

QUESTION 10 (10 marks)

Source: CSC429-2023-02, page [3]

Unofficial model answer:

© Hak Cipta Universiti Teknologi MARA

a) SRAM vs DRAM:

i) Bit storage: SRAM stores each bit in a bistable flip-flop (typically a 4- or 6-transistor latch) that holds its state as long as power is applied; no capacitor is involved. DRAM stores each bit as charge on a tiny capacitor (one transistor plus one capacitor per cell), so the bit is represented by the presence or absence of charge rather than by an active latch.

ii) Speed: SRAM is much faster (access times on the order of a few nanoseconds, no refresh overhead, no sense-and-restore cycle). DRAM is slower (tens of nanoseconds) because reading is destructive and requires sense amplifiers plus a write-back/restore, and because of refresh cycles.

iii) Location/use: SRAM is used for cache memory (L1/L2/L3) and registers/small fast buffers close to the CPU. DRAM is used for main memory (the large, cheaper working store), e.g. DDR SDRAM modules.

b) DRAM needs refreshing because each bit is stored as charge on a capacitor that leaks away through the transistor and parasitic paths. Without periodic restoration the charge would decay and the data would be lost. Therefore the memory controller must periodically read and rewrite (refresh) every row within the retention-time window (typically every few milliseconds), which is why DRAM is volatile and has refresh overhead.

c) SRAM would be placed higher in the memory hierarchy (closer to the CPU). Justification: the hierarchy is ordered by speed and cost per bit, with the fastest, most expensive, smallest memories at the top. SRAM is faster, has no refresh overhead, and consumes less dynamic power per access, but it is larger per bit (more transistors) and hence more expensive, so it is used in small caches near the CPU. DRAM is slower and cheaper per bit, so it sits lower, as main memory.

Marking points:

- *1 mark: SRAM uses a flip-flop/latch (4-6 transistors) to hold a bit; DRAM uses a capacitor (1T1C) storing charge.*
- *1 mark: SRAM is faster (no refresh, no restore); DRAM is slower (sense/restore and refresh overhead).*
- *1 mark: SRAM used in cache/registers near CPU; DRAM used as main memory.*
- *1 mark: DRAM bit is charge on a capacitor that leaks.*
- *1 mark: refresh periodically reads and rewrites each row to restore charge before it decays.*
- *1 mark: correct statement that DRAM is volatile / refresh overhead exists.*
- *1 mark: SRAM placed higher in the hierarchy (closer to CPU).*
- *1 mark: justification by speed (SRAM faster).*
- *1 mark: justification by cost per bit / density (SRAM more expensive, less dense, so small caches).*
- *1 mark: coherent overall comparison linking hierarchy ordering to speed and cost.*

Model confidence: high.

Model note: Unofficial model answer; not an official UTM marking scheme. No figure/table was supplied, so the answer relies only on the question text. Mark allocation is indicative and assumes roughly 3 marks for (a), 3 for (b) and 4 for (c).

© Hak Cipta Universiti Teknologi MARA

PART B — MARKING GUIDE

QUESTION 11 (20 marks)

Source: CSC429-2013-12, page [6]

Unofficial model answer:

Subpart (a) — One-address instructions. In a one-address machine every arithmetic instruction implicitly uses an accumulator (here Register X), and the single operand is a memory location. The equation in Figure 4 is taken to be X = (A + B) × (C − D) ÷ (E + F); the figure itself was supplied only as a cropped image, so this expression is an assumption. Program: `LOAD A` (X ← A); `ADD B` (X ← X + B = A + B); `STORE T1` (save A + B); `LOAD C` (X ← C); `SUB D` (X ← C − D); `STORE T2` (save C − D); `LOAD T1` (X ← A + B); `MUL T2` (X ← (A+B)(C−D)); `STORE T3`; `LOAD E` (X ← E); `ADD F` (X ← E + F); `STORE T4`; `LOAD T3` (X ← numerator); `DIV T4` (X ← (A+B)(C−D)/(E+F)); `STORE X` (final answer in X). Temporary memory locations T1–T4 are needed because only one operand can be addressed per instruction.

Subpart (b) — Two-address instructions. A two-address instruction has the form OP (X ← Y op Z) or OP (X ← X op Y), destroying one source. Using the register-to-register forms from Table 2: `MOVE X, A` (X ← A); `ADD X, B` (X ← A + B); `MOVE Y, C` (Y ← C); `SUB Y, D` (Y ← C − D); `MUL X, Y` (X ← (A+B)(C−D)); `MOVE Z, E` (Z ← E); `ADD Z, F` (Z ← E + F); `DIV X, Z` (X ← (A+B)(C−D)/(E+F)). The final result is already in X, so no STORE is required. Only the temporary registers X, Y and Z are used; A–F are never overwritten.

Subpart (c) — Three-address instructions. A three-address instruction specifies two sources and a distinct destination, OP (X ← Y op Z), so no intermediate results need to be saved and no source is destroyed. Program: `ADD X, A, B` (X ← A + B); `SUB Y, C, D` (Y ← C − D); `MUL X, X, Y` (X ← (A+B)(C−D)); `ADD Z, E, F` (Z ← E + F); `DIV X, X, Z` (X ← (A+B)(C−D)/(E+F)). The answer ends in X as required. This is the shortest program, illustrating that three-address format minimises instruction count at the cost of a longer instruction encoding.

Marking points:

• 2 marks: correct identification/statement of the expression to be computed from Figure 4 (e.g. X = (A+B)(C−D)/(E+F))

• 6 marks: one-address program — correct use of LOAD/STORE/ADD/SUB/MUL/DIV with accumulator X, correct temporary storage of partial results, correct order of operations, final result in X (deduct for missing STORE or wrong operand order)

• 5 marks: two-address program — correct MOVE/ADD/SUB/MUL/DIV (X←Y op Z or X←X op Y) forms, correct use of temporary registers X, Y, Z, A–F unmodified, final result in X

• 5 marks: three-address program — correct OP (X←Y op Z) form, minimal instruction sequence, A–F unmodified, final result in X

• 2 marks: correct assembly notation and explanatory comments throughout; no modification of registers A–F

Model confidence: low.

Model note: Figure 4 was supplied only as a cropped image with no readable expression; the equation X = (A+B)(C−D)/(E+F) is assumed from typical past-paper style and must be checked against the actual figure. Table 2 text is garbled (e.g. 'MOVE (X e Y)', 'ADD (X k.. X + Y)'), so instruction mnemonics/operand orders are interpreted as standard one-, two- and three-address forms. This is an unofficial model answer, not an official UTM marking scheme.

END OF MARKING SCHEME

© Hak Cipta Universiti Teknologi MARA